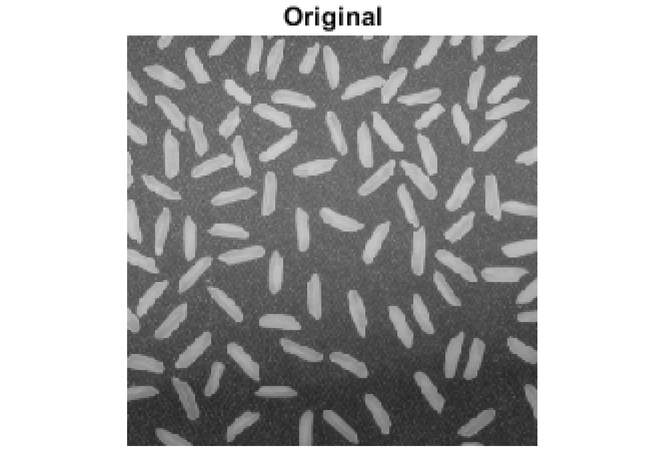
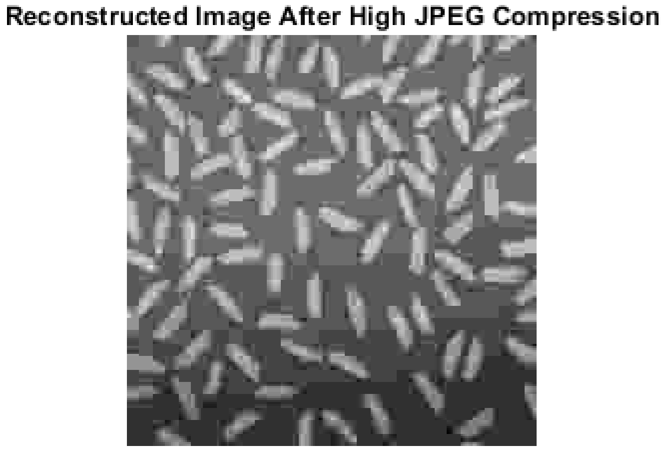
```
clc;
clear all;
close all;

I=imread("rice.png");
if size(I,3)==3
    I=rgb2gray(I);
end
I=double(I);

Q=[16 11 10 16 24 40 51 61;
   12 12 14 19 26 58 60 55;
   14 13 16 24 40 57 69 56;
   14 17 22 29 51 87 80 62;
   18 22 37 56 68 109 103 77;
   24 35 55 64 81 104 113 92;
   49 64 78 87 103 121 120 101;
   72 92 95 98 112 100 103 99];
%Defining the standard JPEG quantization matrix.

factor=10;
Q=Q*factor;
blockSize=8;
[m,n]=size(I);
reconstructed=zeros(m,n);

for i=1:blockSize:(m-blockSize+1)
    for j=1:blockSize:(n-blockSize+1)

        block=I(i:i+7,j:j+7);            %Extracting one valid 8x8
 block from the image.
        block=block-128;                  %Shifting pixel values
 around zero to improve DCT efficiency.
        dctBlock=dct2(block);            %Applying Discrete Cosine
 Transform to convert block into frequency domain.
        quantBlock=round(dctBlock./Q);    %Quantizing the DCT
 coefficients which causes lossy compression.
        dequantBlock=quantBlock.*Q;       %Reversing quantization step
 for decompression during reconstruction.
        idctBlock=idct2(dequantBlock);    %Applying inverse DCT to
 bring the frequency block back to spatial form.
        idctBlock=idctBlock+128;          %Shifting pixel values back
 to the original intensity range.
        reconstructed(i:i+7,j:j+7)=idctBlock;  %Placing the
 reconstructed block back into the output image.

    end
end

reconstructed=uint8(reconstructed);
```

```
figure;
imshow(uint8(I));
title("Original");


figure;
imshow(reconstructed);
title("Reconstructed Image After High JPEG Compression");
%Displaying the decompressed output image after DCT compression.
```





Published with MATLAB® R2020b